05 · The dashboard: how `app/app.py` works

Transcript Intelligence: notebook 5 of 5

What this is. A guided tour of `app/app.py`, the Streamlit dashboard. The notebook imports the dashboard's own functions and runs them on the real data, so every chart and number below is exactly what the dashboard shows, not a re-implementation.

The idea in one line. *Precompute, then present.* The pipeline (`src/pipeline.py`) does all the heavy and paid work: loading, enrichment, Claude classification and sentiment scoring. The dashboard only reads those files. What depends on the sidebar filters (account risk, the incident timeline, SLA, competitors, talk time) is cheap, so it's recomputed from the filtered calls on every interaction, and every number follows the selected time period.

How to run the dashboard. `streamlit run app/app.py`, or `python run_app.py`, or `TranscriptIntel.exe`. This notebook doesn't start it and makes no LLM calls.

Contents

1. Setup: importing the dashboard without starting it
2. Code map
3. Loading data: modes and caching
4. Filters and the time period
5. The context: one dict that feeds every tab
6. Overview tab
7. Topics tab
8. Confidence tab
9. Accounts tab
10. Incidents tab
11. Insights tab
12. Tool Telemetry tab
13. Every number follows the period

14. New transcripts, refresh and logs
15. Extending the dashboard

1. Setup: importing the dashboard without starting it

`streamlit run` executes `app.py` as `__main__`, and only then does the last line call `main()`. Imported under another name, the module just defines its functions. Streamlit calls made outside a running app (such as `st.cache_data`) fall back to a simple in-memory mode and log warnings, which are silenced here.

In [1]:
```
%load_ext autoreload
%autoreload 2

import sys, json, logging, importlib.util, inspect
from datetime import timedelta
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(ROOT))

import pandas as pd
import plotly.io as pio
from IPython.display import HTML, Markdown, display

import streamlit  # noqa: F401  (create Streamlit's loggers first so they can be silenced)
for name in list(logging.root.manager.loggerDict):
    if name.startswith("streamlit"):
        logging.getLogger(name).setLevel(logging.ERROR)

spec = importlib.util.spec_from_file_location("dashboard", ROOT / "app" / "app.py")
app = importlib.util.module_from_spec(spec)
spec.loader.exec_module(app)

pio.renderers.default = "plotly_mimetype+notebook_connected"
pd.set_option("display.max_columns", 50)
pd.set_option("display.max_colwidth", 120)
print("Imported", spec.origin)
```

```
Imported C:\dev\transcript_intel\app\app.py
```

2. Code map

`app.py` is one file, organised top to bottom in the order the page is built:

Section	Functions	Job
Constants	`CALL_TYPE_COLORS`, `TIER_COLORS`, `SCAN_INTERVALS`, `DATE_FMT`, …	Fixed colours per call type, status colours reserved for state, mm/dd/yyyy formats
Loading	`available_modes`, `load_base`, `load_mode`, `merged_calls`	Read the pipeline's files, cached
Header and data	`config_popover`, `run_refresh`, `transcript_watch`, `data_section`	⚙ Config, the new-transcript check, the Refresh button and the pipeline log
Filters	`quick_ranges`, `sidebar_filters`, `filter_calls`	Time period, call type, product and account
Helpers	`kpi`, `show_table`, `bar_list`, `badge`, `money`, `fmt_ts`, …	Small HTML pieces and table formatting
Narrative	`headline_insight`, `accomplishments`, `improvements`	The Overview's sentences, generated from the data
Charts	`*_chart`, `account_heatmap`, `theme_period_heatmap`	One Plotly figure each, no Streamlit calls inside
Tabs	`render_overview` … `render_telemetry`, `account_brief`, the two dialogs	Lay out each tab from the context
Entry point	`build_context`, `main`	Load, filter, compute the context, draw the tabs

The chart and narrative functions take plain DataFrames and return a figure or a string. That separation is what lets this notebook reuse them.

In [2]:
```
own = {n: f for n, f in inspect.getmembers(app, inspect.isfunction) if f.__module__ == "dashboard"}
def kind(n):
    if n.startswith("render_") or n.endswith("_dialog"): return "tab"
    if n.endswith("_chart") or n.endswith("_heatmap"): return "chart"
    if n in {"headline_insight", "accomplishments", "improvements", "account_brief"}: return "narrative"
    return "other"
pd.DataFrame([{"function": n, "kind": kind(n), "line": inspect.getsourcelines(f)[1],
```

```
                "does": (inspect.getdoc(f) or "").split("\n")[0]} for n, f in own.items()]) \
  .sort_values("line").reset_index(drop=True)
```

Out[2]:

	function	kind	line	does
0	available_modes	other	122	
1	merged_calls	other	141	
2	_as_list	other	147	
3	config_popover	other	154	
4	has_llm_key	other	176	
...	...	...	...	...
57	render_incidents	tab	1223	
58	render_insights	tab	1294	
59	render_telemetry	tab	1342	
60	build_context	other	1437	Everything the tabs show, recomputed for the filtered calls and period.
61	main	other	1469	

62 rows × 4 columns

3. Loading data: modes and caching

Two groups of files are read:

- `load_base()` : tables that don't depend on the analysis mode (calls, sentences, key moments, action items, staff roster, sentiment scores), plus the time of the last pipeline run. That time doubles as a **data version**: if it changes on disk, the sidebar offers *Load latest data*.
- `load_mode(mode)` : the outputs that differ between **Hybrid (LLM + rules)** and **Rules only**: themes, topic clusters, the incident register and the insight tables, from `data/processed/<mode>/` .

Both use `@st.cache_data`, so files are read once per pipeline run, not on every click. A refresh clears the cache. These files are written after the pipeline's screening stage, so non-business calls (birthdays, farewells, socials) are already gone: the dashboard has no filter for them because it never sees them (`01_explore`, section 4).

In [3]:
```
modes = app.available_modes()
print("Modes built:", {m: app.MODES[m] for m in modes})

tables, call_sent, sent_metrics, version = app.load_base()
MODE = "hybrid" if "hybrid" in modes else modes[0]
call_topics, clusters, topic_metrics, register, ins = app.load_mode(MODE)
all_calls = app.merged_calls(tables, call_sent, call_topics)

print(f"Data version (last pipeline run): {version}")
pd.DataFrame({"rows": {k: len(v) for k, v in tables.items()}})
```

```
Modes built: {'hybrid': 'Hybrid (LLM + rules)', 'rules': 'Rules only'}
Data version (last pipeline run): 2026-10-29T19:50:54+00:00
```

Out[3]:

	rows
calls	115
sentences	4778
key_moments	490
action_items	466
staff_roster	27

In [4]:
```
# One row per call: enrichment + themes + sentiment, the table every tab starts from
print(all_calls.shape)
all_calls[["date", "call_type", "account", "title", "primary_theme", "sentiment_score",
           "customer_vader", "arc", "teams"]].head()
```

```
(115, 57)
```

Out[4]:

	date	call_type	account	title	primary_theme	sentiment_score	customer_vader	arc	teams
0	2026-02-03	support	Vanta	Support Case #3286 - Vanta Health Systems API Integration Error	Backup / Protect	2.4	0.230469	improved	[Support, Vanta (customer)]
1	2026-02-04	external	Coastal	Aegis / Coastal Living Co - Onboarding Kickoff	Onboarding & deployment	4.9	0.621488	flat	[Customer Success & Sales, Coastal (customer)]
2	2026-02-05	internal	NaN	Detect Team - Sprint Planning	Internal planning	3.6	NaN	flat	[Engineering, Solutions Engineering]
3	2026-02-06	support	Summit	Support Case #9279 - Summit Trust Billing Inquiry	Billing disputes	3.8	0.392428	improved	[Support, Summit (customer)]
4	2026-02-07	external	Blackridge	Aegis / Blackridge Investments - Renewal Discussion	Renewal & contracts	4.7	0.712802	declined	[Customer Success & Sales, Solutions Engineering, Blackridge (customer)]

4. Filters and the time period

`sidebar_filters()` draws the sidebar widgets and hands their values to `filter_calls()`, which does the actual filtering. The quick ranges are derived from the timeline in `src/enrich.py` (outage start and end, Comply v2 launch), clipped to the data.

Two details matter for new data. When a refresh brings in calls outside the old range, a quick range moves with the data and a custom range is clipped. Dates are shown, and picked, as mm/dd/yyyy.

In [5]:
```
first, last = all_calls["start_time"].min().date(), all_calls["start_time"].max().date()
ranges = app.quick_ranges(first, last)
pd.DataFrame([{"quick range": k, "from": a.strftime(app.DATE_FMT), "to": b.strftime(app.DATE_FMT),
               "calls": len(app.filter_calls(all_calls, a, b))} for k, (a, b) in ranges.items()])
```

Out[5]:

	quick range	from	to	calls
0	All time	02/03/2026	07/01/2026	115
1	Pre-outage	02/03/2026	03/09/2026	34
2	Outage (03/10/2026–03/18/2026)	03/10/2026	03/18/2026	17
3	Recovery	03/19/2026	04/06/2026	17
4	Post-Comply-launch	04/07/2026	07/01/2026	47
5	Last 30 days	06/02/2026	07/01/2026	7

In [6]:
```
# The same function the sidebar uses: e.g. support calls about Protect during the outage
a, b = ranges[next(k for k in ranges if k.startswith("Outage"))]
app.filter_calls(all_calls, a, b, types=["support"], product="Protect")[["date", "account", "title", "sentiment_score"]]
```

Out[6]:

	date	account	title	sentiment_score
47	2026-03-16	Silverline	Support Case #3103 - Silverline Brands Detect False Positives After Patch	2.4

5. The context: one dict that feeds every tab

`build_context()` takes the filtered calls and computes everything the tabs need, using the same functions as the pipeline (`account_health`, `incident_timeline`, `competitor_mentions`, `talk_time`, `monthly_availability`). That is how account risk, SLA and the rest follow the period.

Below it's built for **All time** with the default 99.5% SLA target. Change `START` / `END` (or `TYPES`, `PRODUCT`, `ACCOUNT`) and re-run the rest of the notebook to see any other view.

In [7]:
```
START, END = ranges["All time"]
TYPES, PRODUCT, ACCOUNT = app.CALL_TYPES, "All products", ""
SLA = app.SLA_TARGET

calls = app.filter_calls(all_calls, START, END, TYPES, PRODUCT, ACCOUNT)
ctx = app.build_context(tables, register, ins, calls, START, END, SLA)
```

```
pd.DataFrame([{"key": k, "type": type(v).__name__, "size": len(v) if hasattr(v, "__len__") else v}
              for k, v in ctx.items()])
```

Out[7]:

	key	type	size
0	calls	DataFrame	115
1	start	date	2026-02-03
2	end	date	2026-07-01
3	km	DataFrame	490
4	ai	DataFrame	466
5	sla_target	float	0.995
6	health	DataFrame	33
7	timeline	DataFrame	51
8	incident_metrics	dict	10
9	comp	DataFrame	116
10	talk	DataFrame	80
11	reps	DataFrame	15
12	register	DataFrame	33
13	register_all	DataFrame	33
14	sla	DataFrame	6
15	credit_calls	set	8
16	asks	DataFrame	8
17	gap_moments	DataFrame	63

6. Overview tab

The Overview opens with a **headline** and two lists, **accomplishments** and **areas to improve**. None of this text is written by hand or by an LLM: it's generated by rules from the context, so it changes with the period. For example, choose *Pre-outage* and the outage sentence disappears.

In [8]:
```
display(HTML(f"<p style='font-size:1.05em'>{app.headline_insight(calls, START, END, ctx['sla'])}</p>"))
display(HTML("<b>Accomplishments</b><ul>" + "".join(f"<li>{t}</li>" for t in
            app.accomplishments(calls, ctx["km"], ctx["register"], ctx["health"])) + "</ul>"))
display(HTML("<b>Areas to improve</b><ul>" + "".join(f"<li>{t}</li>" for t in
            app.improvements(calls, ctx["register"], ctx["health"], ctx["sla"], ctx["comp"], ctx["asks"])) + "</ul>"))
```

The Detect outage (03/10/2026–03/18/2026) pulled confidence to **2.18** from 3.78 before it, and came up in 51 of 115 calls in this period. It breached the 99.5% SLA in Mar 2026 (6.0 h down vs 3.7 h allowed). Since then confidence is back to 3.56; **support calls are still the lowest at 3.02.**

Accomplishments

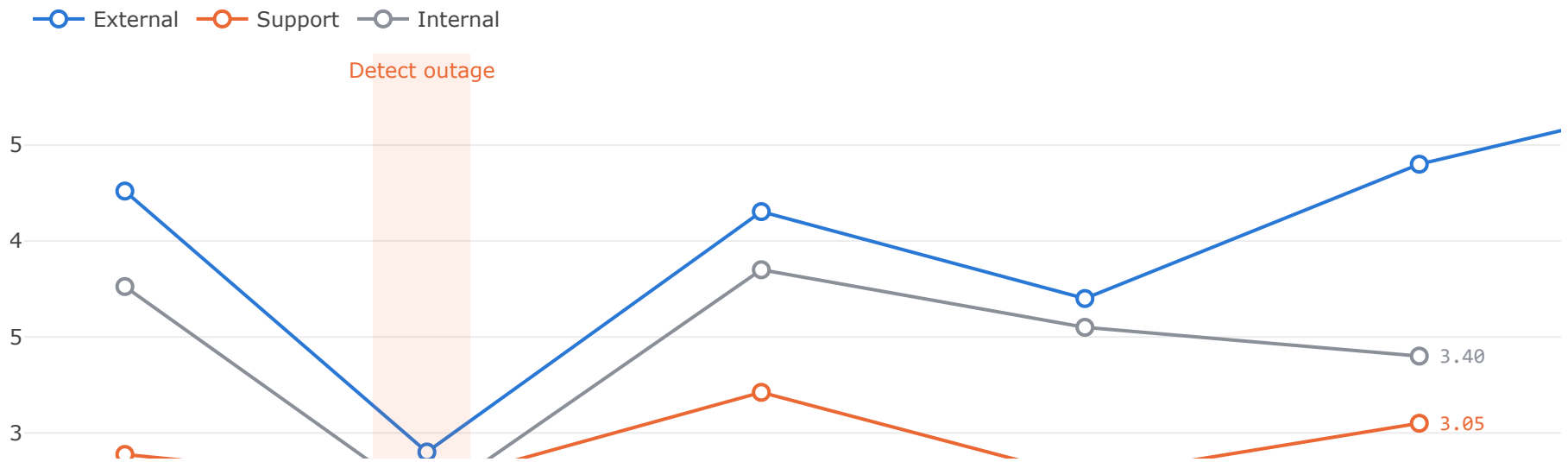
- **8 renewal / contract conversations ended positively** (Atlas, Axiom, Blackridge, Bridgeport).
- **Comply v2 launch landed well**: average confidence 3.76 across 24 calls since GA on 04/07/2026.
- **7 incidents resolved**, including the Detect Pipeline Failure (restored 03/18/2026 16:51 UTC).
- **49% of calls ended better than they started**, so teams are turning difficult conversations around.
- **7 accounts trending up**: Cobalt, Trailhead, Vanta, Forge.

Areas to improve

- **Support confidence trails at 2.93** (vs 3.75 for external calls).
- **10 high-risk accounts**: Summit, Silverline, Ridgeline, Meridian — churn signals, competitors or falling confidence.
- **9 P1 incidents still open**, e.g. Vanta Health Systems API Integration Error.
- **SLA breached in Mar 2026** — 6.0 h of platform downtime; SLA credits are being negotiated.
- **Competitive pressure**: SentinelShield named on 28 calls.

Confidence by call type over time. Weekly points for periods up to 45 days, monthly otherwise (`bucket()`), with the outage window shaded.

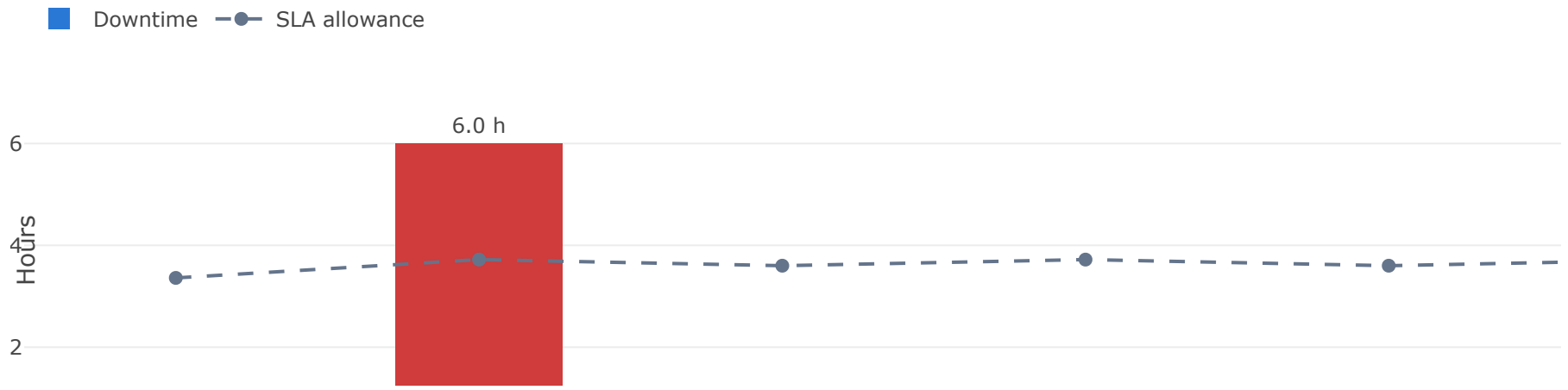
In [9]:
```
app.confidence_by_type_chart(calls, START, END)
```



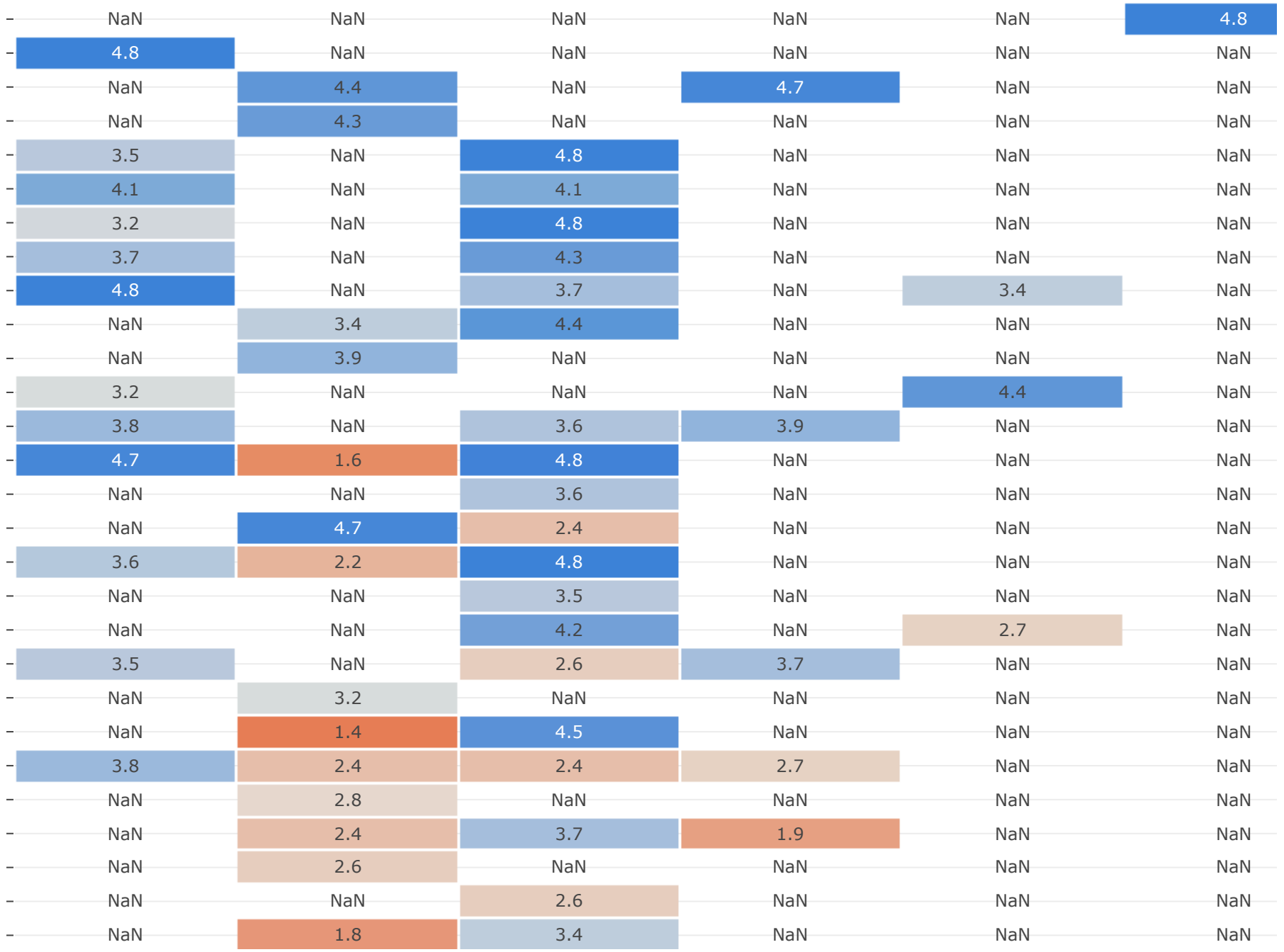
Outage duration and SLA impact by month. Downtime comes from the incident register; the allowance is the month's hours × (1 − SLA target). A breached month is drawn in the critical colour.

In [10]:
```
display(ctx["sla"][["label", "outage_hours", "allowed_hours", "pct_time_down", "sla_breached", "incidents"]])
app.sla_chart(ctx["sla"])
```

	label	outage_hours	allowed_hours	pct_time_down	sla_breached	incidents
0	Feb 2026	0.0	3.36	0.000000	False	
1	Mar 2026	6.0	3.72	0.008065	True	INC-20260310
2	Apr 2026	0.0	3.60	0.000000	False	
3	May 2026	0.0	3.72	0.000000	False	
4	Jun 2026	0.0	3.60	0.000000	False	
5	Jul 2026	0.0	3.72	0.000000	False	



In [11]:
```
app.account_heatmap(calls, START, END)
```


NaN NaN NaN NaN NaN 4.8
4.8 NaN NaN NaN NaN NaN
NaN 4.4 NaN 4.7 NaN NaN
NaN 4.3 NaN NaN NaN NaN
3.5 NaN 4.8 NaN NaN NaN
4.1 NaN 4.1 NaN NaN NaN
3.2 NaN 4.8 NaN NaN NaN
3.7 NaN 4.3 NaN NaN NaN
4.8 NaN 3.7 NaN 3.4 NaN
NaN 3.4 4.4 NaN NaN NaN
NaN 3.9 NaN NaN NaN NaN
3.2 NaN NaN NaN 4.4 NaN
3.8 NaN 3.6 3.9 NaN NaN
4.7 1.6 4.8 NaN NaN NaN
NaN NaN 3.6 NaN NaN NaN
NaN 4.7 2.4 NaN NaN NaN
3.6 2.2 4.8 NaN NaN NaN
NaN NaN 3.5 NaN NaN NaN
NaN NaN 4.2 NaN 2.7 NaN
3.5 NaN 2.6 3.7 NaN NaN
NaN 3.2 NaN NaN NaN NaN
NaN 1.4 4.5 NaN NaN NaN
3.8 2.4 2.4 2.7 NaN NaN
NaN 2.8 NaN NaN NaN NaN
NaN 2.4 3.7 1.9 NaN NaN
NaN 2.6 NaN NaN NaN NaN
NaN NaN 2.6 NaN NaN NaN
NaN 1.8 3.4 NaN NaN NaN

7. Topics tab

Themes come from `data/processed/<mode>/call_topics.parquet` (see `02_topics`). The tab shows how often each theme comes up by period, how confident calls about each theme are, and the primary theme by call type.

In [12]:
```
app.theme_counts(calls).to_frame('calls mentioning')
```

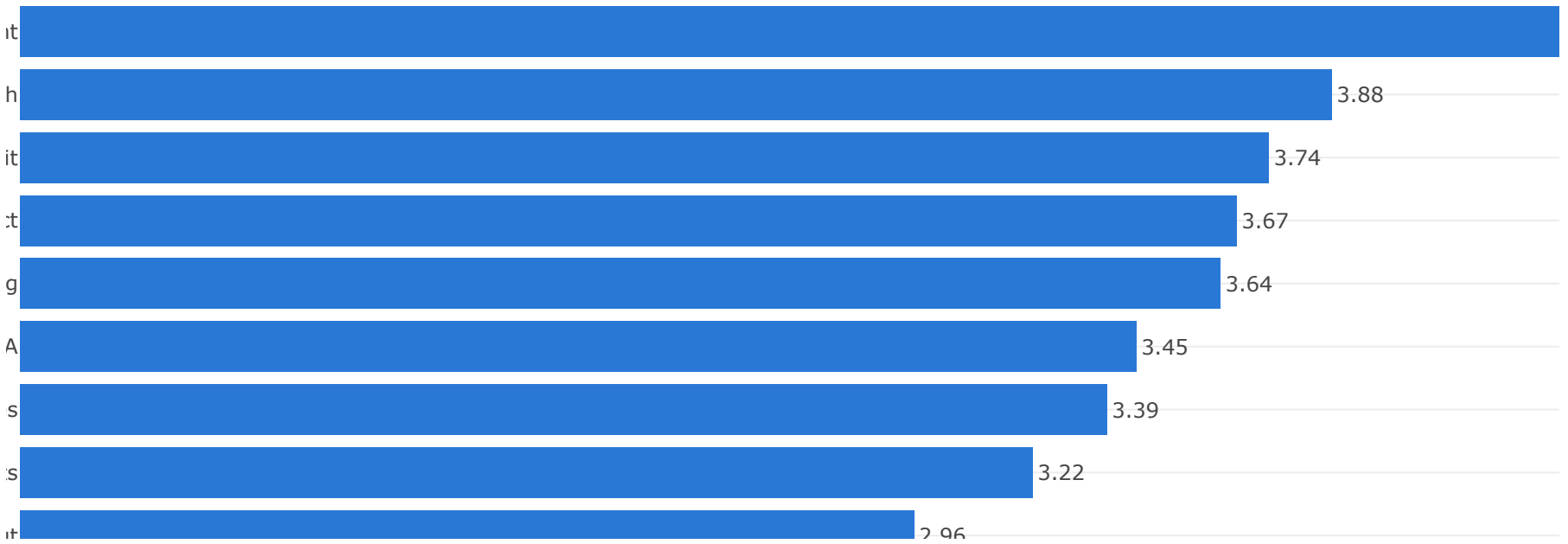
Out[12]:

themes	calls mentioning
Detect outage & fallout	53
Compliance & audit	51
Comply v2 launch	43
Renewal & contracts	31
Competitive threat	29
Internal planning	20
Identity / SSO / MFA	20
Backup / Protect	17
Onboarding & deployment	15
Billing disputes	7

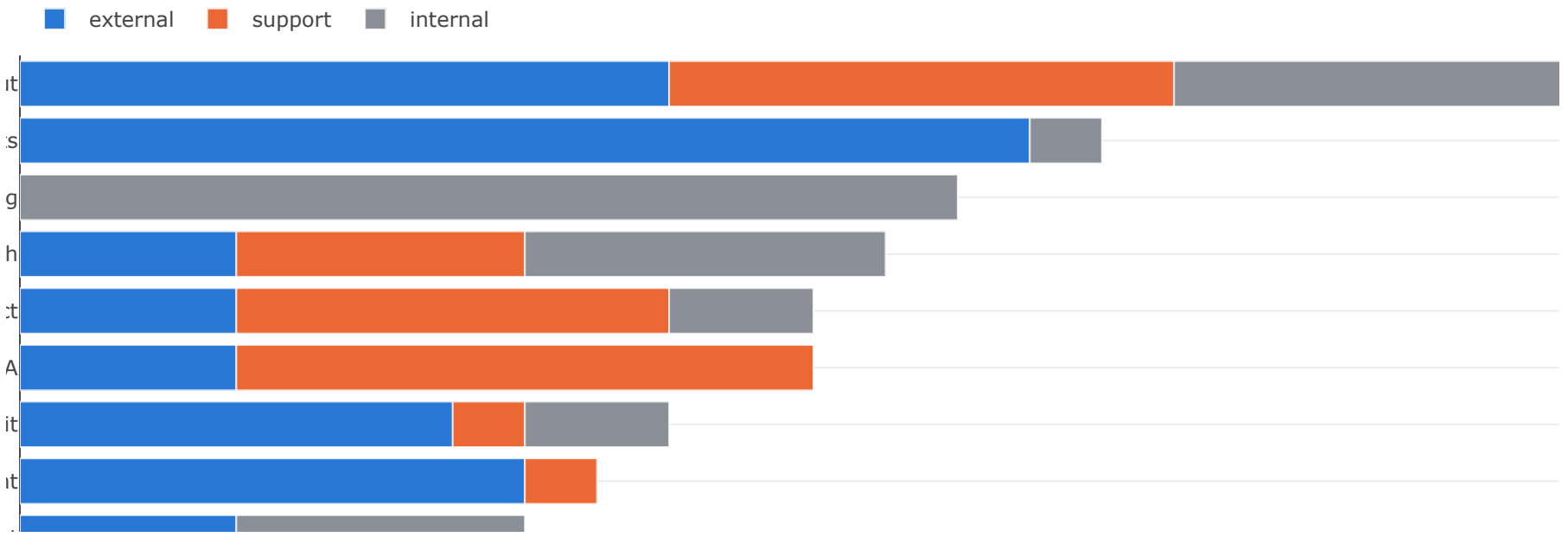
In [13]:
```
app.theme_period_heatmap(calls)
```

16	2	9	
2	17	8	
8	0	11	
6	3	6	
7	0	1	
3	9	6	
7	0	4	
10	0	1	
7	0	1	

In [14]:
```
app.confidence_by_theme_chart(calls)
```



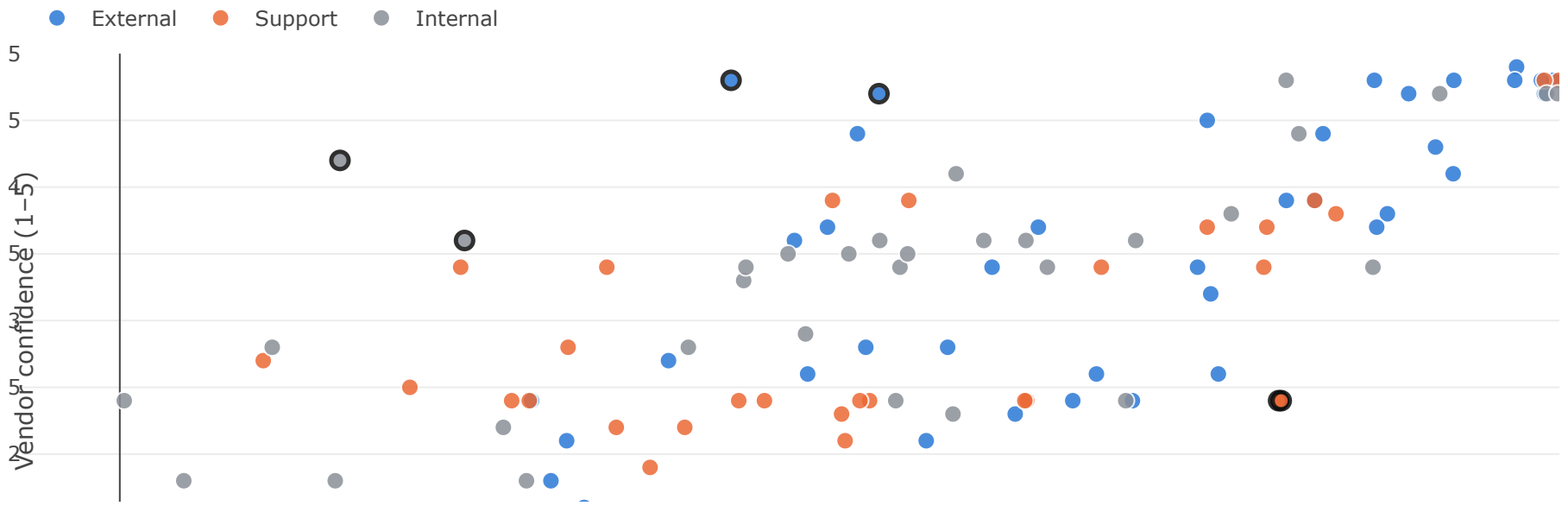
In [15]:
```
app.primary_theme_by_type_chart(calls)
```



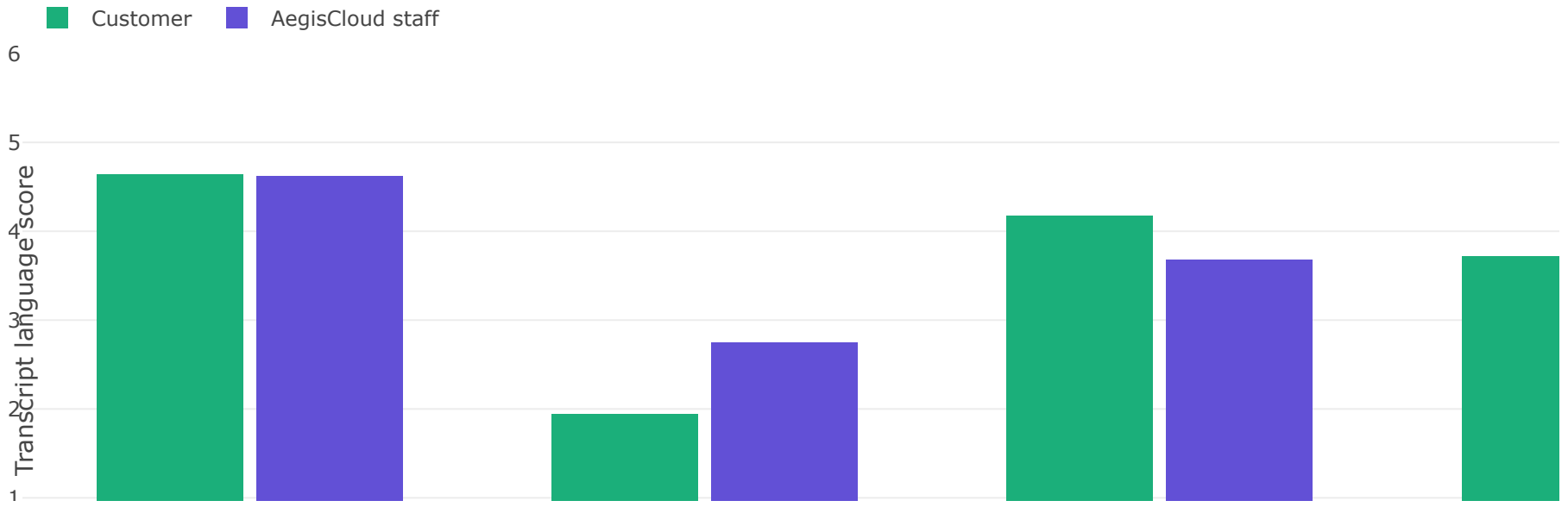
8. Confidence tab

"Confidence" is the vendor's 1–5 call score. The dashboard checks it against its own reading of the transcript text (`03_sentiment`) without naming the scoring library, so the labels stay in plain language.

In [16]:
```
app.vendor_vs_transcript_chart(calls)
```



In [17]:
```
app.side_by_period_chart(calls)
```



In [18]:
```
# The slice selector: any of call type, month, period, product, theme
fig, table = app.slice_chart(calls, "product")
display(table)
fig
```

	value	n_calls	vendor_mean	vader_mean	customer_mean	staff_mean	share_improved	dimension
1	Detect	69	3.293	0.340	0.359	0.344	0.493	product
2	Identity	28	3.550	0.400	0.395	0.402	0.214	product
0	Comply	82	3.571	0.396	0.417	0.398	0.451	product
3	LogVault	7	3.600	0.405	0.369	0.418	0.143	product
4	Protect	32	3.634	0.413	0.432	0.418	0.469	product



In [19]:
```
app.arc_chart(calls)
```

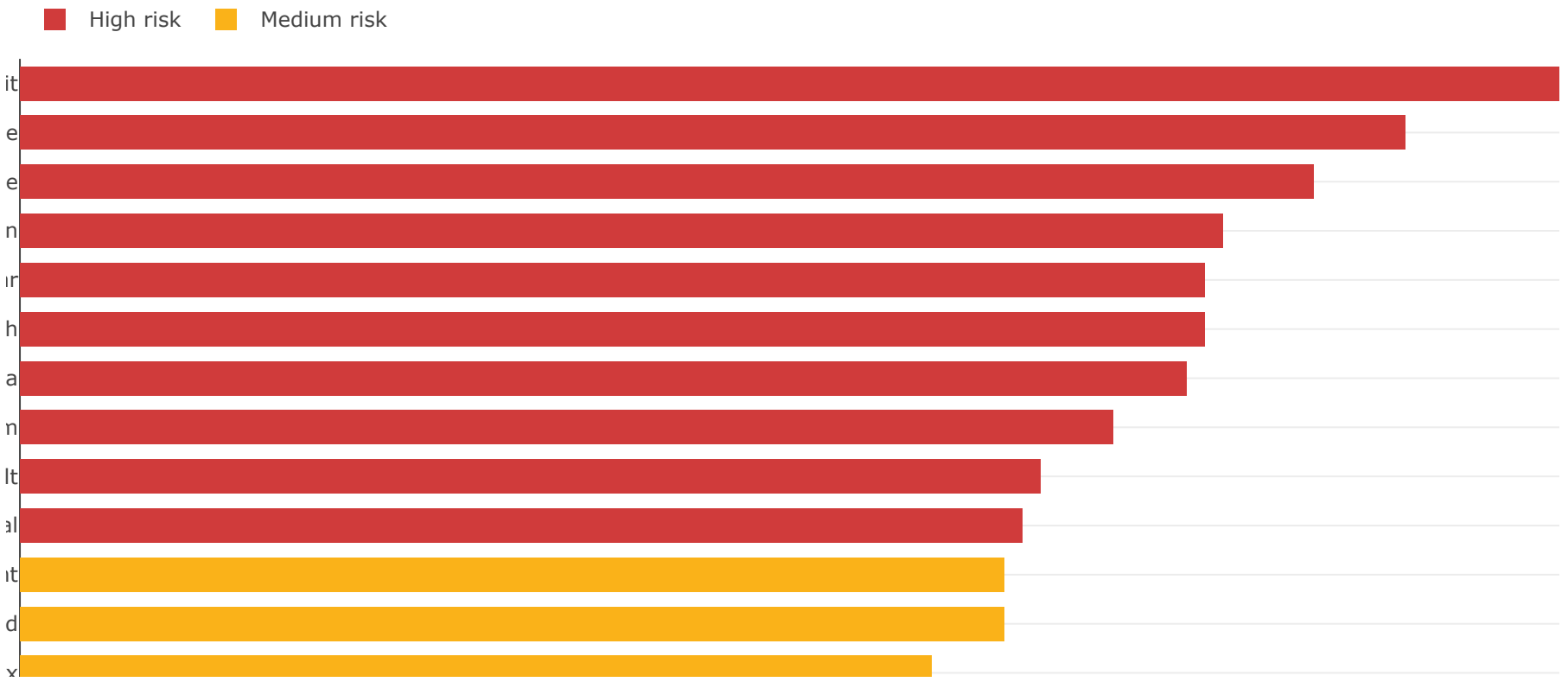


9. Accounts tab

Accounts are ranked by the transparent risk score from `src/insights.py` (see `04_insights`), recomputed for the period. Clicking an account name in the dashboard opens `account_brief()`: a Markdown summary plus the account's action items, which can be copied or downloaded.

In [20]:
```
h = ctx["health"]
display(h["risk_tier"].value_counts().reindex(app.TIERS).to_frame("accounts"))
app.risk_ranking_chart(h)
```

risk_tier	accounts
High	10
Medium	15
Low	8



In [21]:
```
ACCOUNT_NAME = h.iloc[0]["account"]          # the highest-risk account; any name from h works
brief, items = app.account_brief(ACCOUNT_NAME, ctx)
display(Markdown(brief))
items
```

Account brief: Summit

Period 02/03/2026 – 07/01/2026 · generated 10/29/2026 20:04 UTC

Current state

- **Risk:** High (88/100) — 4 churn signals; named VaultEdge; latest call 2.7/5; down 1.1 since first call; 2 support cases; hit by outage
- **Calls:** 4 (2 support), 02/06/2026 → 05/14/2026
- **Confidence:** average 2.83, latest 2.7 (-1.1 since the first call in the period)
- **Churn signals:** 4 · **concerns:** 4
- **Competitors named:** VaultEdge
- **Affected by the Detect outage:** yes · **renewal in play:** yes
- **Latest call:** Aegis / Summit Trust - Renewal Decision

Open incidents (2)

- CASE-9279 · P3 billing · Summit Trust Billing Inquiry (reported 02/06/2026 09:15 UTC, owner Sarah Chen)
- CASE-6147 · P2 degradation · Summit Trust Detect Alert Delays (reported 04/15/2026 12:30 UTC, owner David Kim)

Action items (16)

- ☐ **Sarah Chen:** Submit billing adjustment request to finance team for the 200-seat overage on invoice INV-2026-00847 and send reference number to Gregory *(02/06/2026, Support Case #9279 - Summit Trust Billing Inquiry)*
- ☐ **Sarah Chen:** Email Gregory the knowledge base article on bulk deprovisioning and include steps for configuring usage threshold alerts *(02/06/2026, Support Case #9279 - Summit Trust Billing Inquiry)*
- ☐ **Gregory Fisk:** Have IT team bulk deprovision the ~200 test accounts created during the December migration before the February billing period closes *(02/06/2026, Support Case #9279 - Summit Trust Billing Inquiry)*
- ☐ **Gregory Fisk:** Configure usage threshold alerts in the admin console under Account Settings > Billing and Usage *(02/06/2026, Support Case #9279 - Summit Trust Billing Inquiry)*

- [ ] **Maria Santos:** Escalate the open MFA ticket today and provide Alicia with a real status update by end of day *(03/26/2026, Aegis / Summit Trust - Platform Concerns Discussion)*
- [ ] **Maria Santos:** Schedule a technical Identity specialist call for next week and deliver a written summary with accountable action items, owners, and timelines afterward *(03/26/2026, Aegis / Summit Trust - Platform Concerns Discussion)*
- [ ] **Maria Santos:** Send Alicia a formal post-incident report on the March Detect outage and arrange an engineering briefing for her CISO on the architecture changes *(03/26/2026, Aegis / Summit Trust - Platform Concerns Discussion)*
- [ ] **Maria Santos:** Develop a formal remediation plan for all Aegis Identity issues including MFA enforcement, session timeout behavior, and support response quality *(03/26/2026, Aegis / Summit Trust - Platform Concerns Discussion)*
- [ ] **David Kim:** Submit emergency infrastructure rebalance request (IR-8842) to migrate Summit Trust tenant to a less-loaded node cluster and monitor completion *(04/15/2026, Support Case #6147 - Summit Trust Detect Alert Delays)*
- [ ] **David Kim:** Deliver formal written incident summary covering the March outage, latency root cause, and resolution to Alicia, Gregory, and their account manager by EOB tomorrow *(04/15/2026, Support Case #6147 - Summit Trust Detect Alert Delays)*
- [ ] **David Kim:** Escalate the March outage communication failure as a separate internal escalation item and loop in the account manager for upcoming QBR *(04/15/2026, Support Case #6147 - Summit Trust Detect Alert Delays)*
- [ ] **David Kim:** Include documentation on setting up a synthetic LogVault-based latency monitoring workaround in the follow-up email, and flag native latency alerting as a product feedback gap *(04/15/2026, Support Case #6147 - Summit Trust Detect Alert Delays)*
- [ ] **Kevin O'Brien:** Apply the Identity 3.6 MFA enforcement configuration for the loan origination system with Gregory's team Thursday at 9 AM *(05/14/2026, Aegis / Summit Trust - Renewal Decision)*
- [ ] **Kevin O'Brien:** Configure the proxy-side token refresh workaround for SSO session timeouts *(05/14/2026, Aegis / Summit Trust - Renewal Decision)*
- [ ] **Maria Santos:** Send the written renewal proposal tonight including the named support engineer commitment and June timeout fix date *(05/14/2026, Aegis / Summit Trust - Renewal Decision)*
- [ ] **Alicia Monroe:** Present the proposal and this week's results to the Summit Trust vendor committee next Thursday *(05/14/2026, Aegis / Summit Trust - Renewal Decision)*

Risk moments

- Concern — Gregory identifies a 200-seat discrepancy on his January invoice versus his contracted 1,200-seat count
- Technical issue — Root cause identified: ~200 undeprovisioned test accounts from December migration are inflating the billed seat count
- Feature gap — Gregory requests proactive seat-count threshold alerts to avoid discovering overages on future invoices

- Churn signal — Alicia reveals CISO has warned that failure to demonstrate consistent MFA enforcement by end of Q2 will trigger a platform reassessment
- Churn signal — Alicia discloses that her CISO has initiated conversations with competitor VaultEdge, including an upcoming demo, representing an active competitive threat
- Churn signal — Alicia says the vendor committee is leaning toward VaultEdge for identity and possibly monitoring

Recent calls

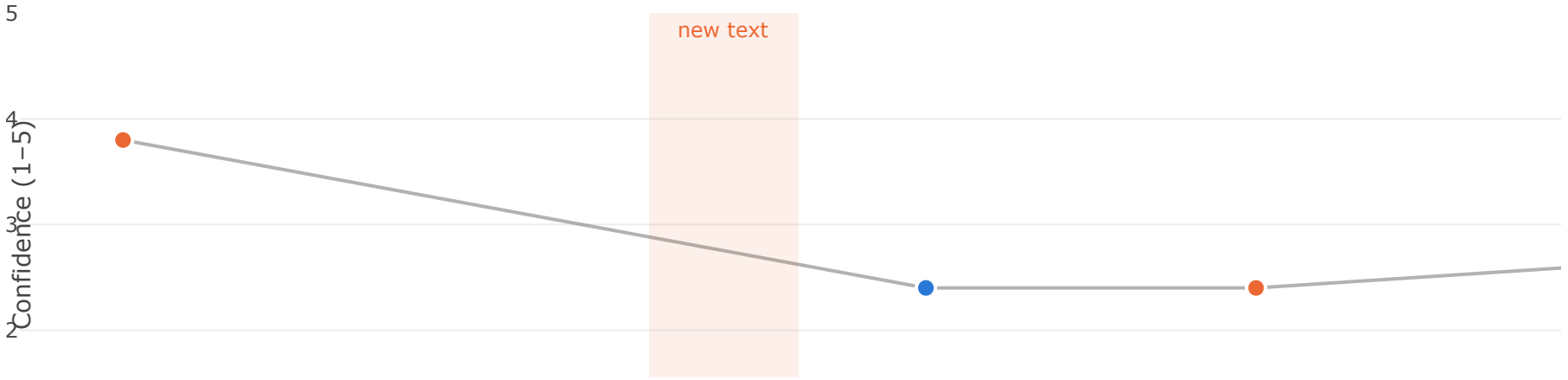
- 05/14/2026 · Aegis / Summit Trust - Renewal Decision · external · confidence 2.7
- 04/15/2026 · Support Case #6147 - Summit Trust Detect Alert Delays · support · confidence 2.4
- 03/26/2026 · Aegis / Summit Trust - Platform Concerns Discussion · external · confidence 2.4
- 02/06/2026 · Support Case #9279 - Summit Trust Billing Inquiry · support · confidence 3.8

Out[21]:

	date	owner	task	from_call
0	2026-02-06	Sarah Chen	Submit billing adjustment request to finance team for the 200-seat overage on invoice INV-2026-00847 and send refere...	Support Case #9279 - Summit Trust Billing Inquiry
1	2026-02-06	Sarah Chen	Email Gregory the knowledge base article on bulk deprovisioning and include steps for configuring usage threshold al...	Support Case #9279 - Summit Trust Billing Inquiry
2	2026-02-06	Gregory Fisk	Have IT team bulk deprovision the ~200 test accounts created during the December migration before the February billi...	Support Case #9279 - Summit Trust Billing Inquiry
3	2026-02-06	Gregory Fisk	Configure usage threshold alerts in the admin console under Account Settings > Billing and Usage	Support Case #9279 - Summit Trust Billing Inquiry
4	2026-03-26	Maria Santos	Escalate the open MFA ticket today and provide Alicia with a real status update by end of day	Aegis / Summit Trust - Platform Concerns Discussion
5	2026-03-26	Maria Santos	Schedule a technical Identity specialist call for next week and deliver a written summary with accountable action it...	Aegis / Summit Trust - Platform Concerns Discussion
6	2026-03-26	Maria Santos	Send Alicia a formal post-incident report on the March Detect outage and arrange an engineering briefing for her CIS...	Aegis / Summit Trust - Platform Concerns Discussion
7	2026-03-26	Maria Santos	Develop a formal remediation plan for all Aegis Identity issues including MFA enforcement, session timeout behavior,...	Aegis / Summit Trust - Platform Concerns Discussion
12	2026-04-15	David Kim	Submit emergency infrastructure rebalance request (IR-8842) to migrate Summit Trust tenant to a less-loaded node clu...	Support Case #6147 - Summit Trust Detect Alert Delays
13	2026-04-15	David Kim	Deliver formal written incident summary covering the March outage, latency root cause, and resolution to Alicia, Gre...	Support Case #6147 - Summit Trust Detect Alert Delays
14	2026-04-15	David Kim	Escalate the March outage communication failure as a separate internal escalation item and loop in the account manag...	Support Case #6147 - Summit Trust Detect Alert Delays
15	2026-04-15	David Kim	Include documentation on setting up a synthetic LogVault-based latency monitoring workaround in the follow-up email,...	Support Case #6147 - Summit Trust Detect Alert Delays
8	2026-05-14	Kevin O'Brien	Apply the Identity 3.6 MFA enforcement configuration for the loan origination system with Gregory's team Thursday at...	Aegis / Summit Trust - Renewal Decision
9	2026-05-14	Kevin O'Brien	Configure the proxy-side token refresh workaround for SSO session timeouts	Aegis / Summit Trust - Renewal Decision

	date	owner	task	from_call
10	2026-05-14	Maria Santos	Send the written renewal proposal tonight including the named support engineer commitment and June timeout fix date	Aegis / Summit Trust - Renewal Decision
11	2026-05-14	Alicia Monroe	Present the proposal and this week's results to the Summit Trust vendor committee next Thursday	Aegis / Summit Trust - Renewal Decision

In [22]:
```
app.account_history_chart(calls[calls["account"] == ACCOUNT_NAME], START, END)
```



10. Incidents tab

The register (from `src/incidents.py`) lists each war-room incident and support case with reported, owned and resolved times. Only incidents reported in the period are shown. The **Call out** button opens a demo dialog that addresses people by handle; nothing is sent or stored.

In [23]:
```
reg = ctx["register"].assign(reported=lambda d: d["reported_at"].map(app.fmt_ts),
                             resolved=lambda d: d["resolved_at"].map(app.fmt_ts))
print(reg["status"].value_counts().to_dict())
reg[["incident_id", "status", "priority", "category", "title", "account", "owner", "reported", "resolved"]].head(10)
```

```
{'Open': 26, 'Resolved': 7}
```

Out[23]:

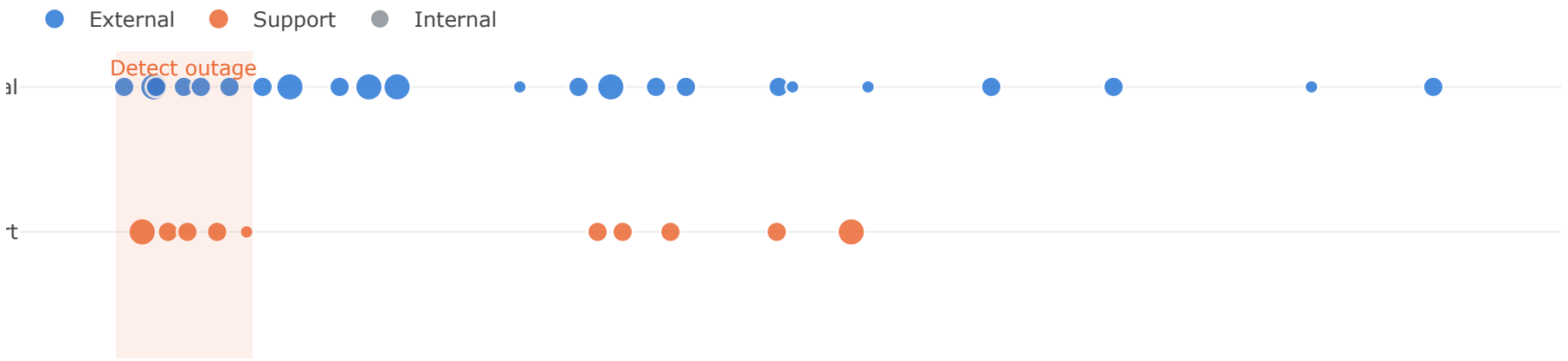
	incident_id	status	priority	category	title	account	owner	reported	resolved
0	CASE-3286	Open	P1	Outage	Vanta Health Systems API Integration Error	Vanta	Priya Patel	02/03/2026 10:00	—
1	CASE-9279	Open	P3	Billing	Summit Trust Billing Inquiry	Summit	Sarah Chen	02/06/2026 09:15	—
2	CASE-3615	Open	P3	Request	Nimbus Platform Custom Report Request	Nimbus	David Kim	02/10/2026 15:30	—
3	CASE-6635	Open	P1	Outage	Nova Retail Group SSO Login Failures	Nova	Elena Vasquez	02/14/2026 13:00	—
4	CASE-1750	Open	P1	Defect	Vanta Health Systems SIEM Connector Issue	Vanta	Sarah Chen	02/18/2026 12:30	—
5	CASE-6977	Open	P2	Degradation	Brightpath Commerce Slow Backup Performance	Brightpath	David Kim	02/21/2026 11:30	—
6	CASE-3677	Open	P1	Outage	Coastal Living Co MFA Token Failures	Coastal	Elena Vasquez	02/24/2026 16:45	—
7	CASE-7572	Open	P3	Billing	Brightpath Commerce Invoice Discrepancy	Brightpath	Marcus Williams	02/27/2026 13:00	—
8	CASE-2638	Open	P2	Defect	Pineridge Systems CloudPrime S3 Backup Connector	Pineridge	Sarah Chen	03/04/2026 10:15	—
9	CASE-3546	Open	P3	Request	Meridian Capital SCIM Provisioning Request	Meridian	Sarah Chen	03/07/2026 13:30	—

In [24]:

```python
# How the Call Out dialog addresses people
open_inc = reg[reg["status"] == "Open"]
[app.handle(n) for n in open_inc["owner"].dropna().unique()[:5]]
```

Out[24]:
```
['@priya.patel',
 '@sarah.chen',
 '@david.kim',
 '@elena.vasquez',
 '@marcus.williams']
```

In [25]:
```
app.incident_chart(ctx['timeline'], START, END)
```



11. Insights tab

Feature asks, competitor intelligence, talk time and action-item workload. Feature-ask counts are recounted from the moments in the period; their names come from the saved clusters.

In [26]:
```
ctx['asks'].head(8)
```

Out[26]:

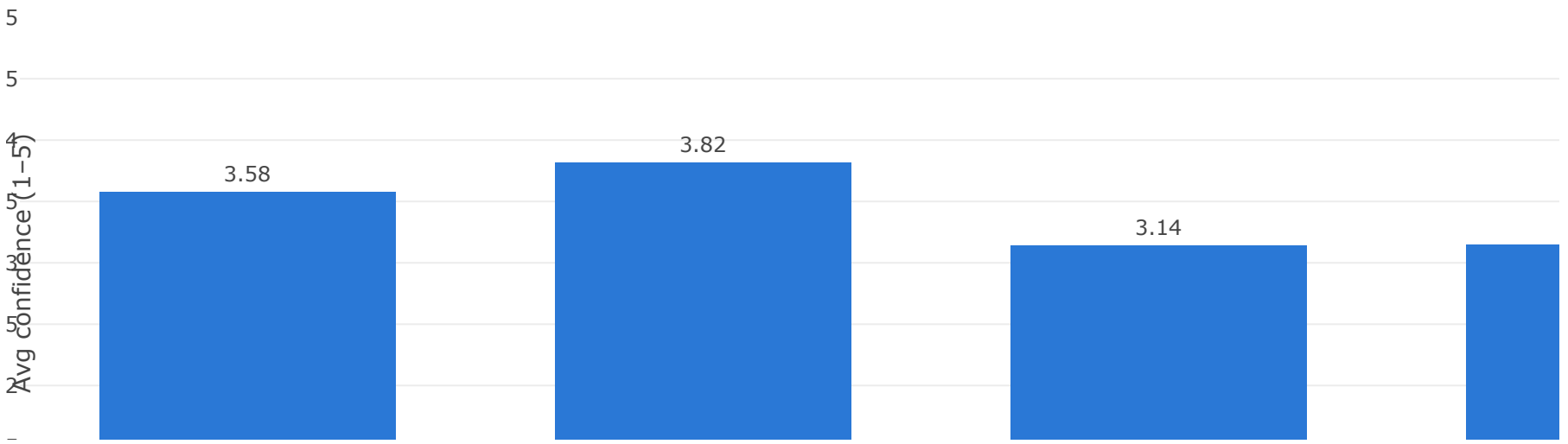
	ask	n_mentions	n_accounts	summary
2	Expanded Compliance Framework Reporting	14	9	Customers want Aegis Comply to offer on-demand, multi-framework compliance reporting that covers more frameworks (SO...
0	Compliance reporting and unified visibility	12	7	Customers want filterable, unified compliance reports and dashboards (HIPAA and SOC 2 together, by control family) p...
4	In-product pipeline health visibility	10	7	Customers want the console to show real-time health, sync latency, and outage status of the monitoring and event ing...
1	Customization and Configurability Gaps	8	7	Customers want more control and flexibility over the product, including custom control labels and compliance templat...
5	Multi-framework compliance mapping	7	7	Customers want compliance reporting that maps controls across SOC 2, ISO 27001, and PCI DSS in a true matrix format,...
6	Proactive Alerting and Cost Visibility	5	3	Customers want proactive alerts and automation, such as seat-count overage warnings, coverage-gap notifications for ...
3	Hierarchical role inheritance	3	1	Customers want nested role inheritance, plus per-role or per-resource session timeout policies, so they can stop dup...
7	Reliable, audit-ready alerting	4	0	Customers want trustworthy threat-monitoring alerts that reduce alert fatigue, proactively warn when telemetry stops...

In [27]:
```
comp = ctx["comp"]
comp.groupby("competitor")["call_id"].nunique().sort_values(ascending=False).to_frame("calls") if len(comp) else "No competito
```

Out[27]:

competitor	calls
SentinelShield	28
VaultEdge	16
CyberNova	8
Fortiguard Pro	5

In [28]:
```
app.talk_bucket_chart(ctx['talk'])
```



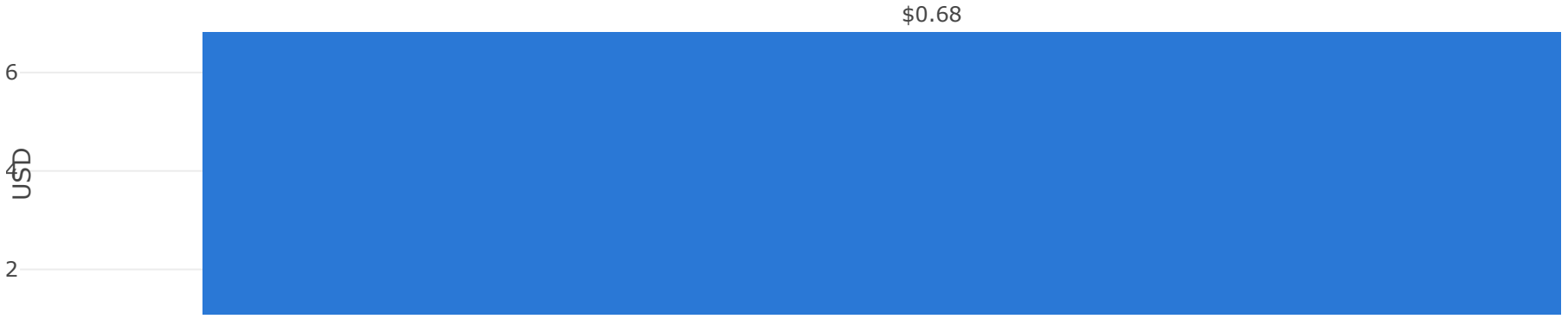
12. Tool Telemetry tab

How well the tooling works and what it costs. LLM spend comes from `data/processed/llm_usage.jsonl`, which `src/llm.py` appends to on every API call (cache hits cost nothing and aren't logged). The month's bar turns red once spend reaches the warning share of the monthly budget, both set in `.env` (`LLM_MONTHLY_BUDGET_USD`, `LLM_BUDGET_WARN_SHARE`).

In [29]:
```
usage = app.load_usage()
print(f"Budget {app.money(app.MONTHLY_BUDGET_USD)}/month, red from {app.BUDGET_WARN_SHARE:.0%}; "
      f"{len(usage)} logged API calls, {app.money(usage['cost_usd'].sum()) if len(usage) else '$0'} in total")
display(usage.groupby("step")["cost_usd"].agg(["count", "sum"]).round(4) if len(usage) else None)
app.monthly_cost_chart(usage, app.MONTHLY_BUDGET_USD, app.BUDGET_WARN_SHARE) if len(usage) else None
```

```
Budget $1,000/month, red from 80%; 231 logged API calls, $0.68 in total
```

step	count	sum
feature_gaps	66	0.1558
incidents	33	0.1680
screen	117	0.2783
topics	15	0.0804



In [30]:
```
pd.Series({k: v for k, v in topic_metrics.items() if not isinstance(v, (dict, list))}, name="topic quality")
```

Out[30]:
```
n_calls                                115
n_llm                                  115
n_rules                                  0
primary_in_rule_themes               0.957
mean_jaccard_llm_vs_rules            0.682
model                      claude-sonnet-5-5
cluster_purity_weighted              0.564
Name: topic quality, dtype: object
```

13. Every number follows the period

`build_context()` is run for each quick range and the main KPIs are put side by side. In the dashboard this is what happens when you change the time period.

In [31]:
```
rows = []
for name, (a, b) in ranges.items():
    c = app.filter_calls(all_calls, a, b)
    if c.empty:
        continue
    x = app.build_context(tables, register, ins, c, a, b, SLA)
    rows.append({
        "period": name, "calls": len(c),
        "avg confidence": round(c["sentiment_score"].mean(), 2),
        "high-risk accounts": int((x["health"]["risk_tier"] == "High").sum()),
        "incidents reported": len(x["register"]),
        "still open": int((x["register"]["status"] == "Open").sum()),
        "outage hours": x["sla"]["outage_hours"].sum(),
        "SLA breached": bool(x["sla"]["sla_breached"].any()),
        "competitor calls": x["comp"]["call_id"].nunique() if len(x["comp"]) else 0,
    })
pd.DataFrame(rows).set_index("period")
```

Out[31]:

period	calls	avg confidence	high-risk accounts	incidents reported	still open	outage hours	SLA breached	competitor calls
All time	115	3.42	10	33	26	6.0	True	38
Pre-outage	34	3.78	1	10	10	6.0	True	7
Outage (03/10/2026–03/18/2026)	17	2.18	4	6	0	6.0	True	9
Recovery	17	3.45	2	4	3	6.0	True	6
Post-Comply-launch	47	3.60	4	13	13	0.0	False	16
Last 30 days	7	3.64	0	2	2	0.0	False	3

What this shows. Risk depends on the window: when this was written, ten accounts were *High* risk over all time but only one in the pre-outage period, because the score only sees that window's calls. The SLA view only counts months inside the period. That's deliberate (the dashboard answers "what happened in this period?"), but worth knowing when comparing screenshots taken with different filters.

14. New transcripts, refresh and logs

The sidebar's Data section runs `transcript_watch()` as a Streamlit **fragment** on a timer (every minute by default; ⚙ Config sets 30 s, 5 min or off). It only **scans and notifies**: it never reloads the page or runs the pipeline by itself, so nobody's view changes under them.

Step	Function	What happens
Scan	`src.pipeline.scan_raw()`	Lists folders in `data/raw/` not yet processed. A folder must be unchanged for 30 s (still copying otherwise), and unreadable ones are reported
Notify	`transcript_watch()`	One pop-up per new batch, plus a sidebar list of the titles; keyword hits are marked *possibly not business*
Ignore	`new_transcript_list()` → `src.screen.exclude_call()`	Adds a new transcript to `data/exclusions.csv` and moves its folder from `data/raw/` to `data/exclude/`; it's never processed
Refresh	`run_refresh()`	On **Refresh dashboard data**: runs the full pipeline (including the non-business screen) under a lock (one run at a time across tabs), shows the log live, clears the cache, reloads
Newer data	data version check	If another tab or the command line produced newer data, offers **Load latest data**
Logs	`src.pipeline.capture_logs()`	Every run is appended to `outputs/logs/pipeline.log`, also shown under **Pipeline log**

In [32]:
```
from src.pipeline import scan_raw, last_run, read_log_tail, PIPELINE_LOG
scan = scan_raw()
run = last_run()
print("New:", scan["new"] or "none", "| arriving:", scan["arriving"] or "none", "| unreadable:", scan["unreadable"] or "none")
print("Possibly not business:", scan["flagged"] or "none", "| excluded so far:", len(app.excluded_ids()))
print("Last run finished:", pd.Timestamp(run["finished_at"]).strftime(app.DATETIME_FMT), "UTC · modes:", run.get("modes_built"
print(f"\nLast lines of {PIPELINE_LOG}:\n" + read_log_tail(8))
```

```
New: none | arriving: none | unreadable: none
Possibly not business: none | excluded so far: 1
Last run finished: 10/29/2026 19:50 UTC · modes: ['rules', 'hybrid']

Last lines of C:\dev\transcript_intel\outputs\logs\pipeline.log:
10/29/2026 12:50:47 INFO ── insights
10/29/2026 12:50:47 WARNING Embedding model unavailable (No module named 'sentence_transformers'); clustering on TF-IDF instead
10/29/2026 12:50:47 INFO Saved 9 insight tables and insights_summary.json to C:\dev\transcript_intel\data\processed\rules
10/29/2026 12:50:47 WARNING Embedding model unavailable (No module named 'sentence_transformers'); clustering on TF-IDF instead
10/29/2026 12:50:54 INFO Saved 9 insight tables and insights_summary.json to C:\dev\transcript_intel\data\processed\hybrid
10/29/2026 12:50:54 INFO    insights done in 7.1s
10/29/2026 12:50:54 INFO ── export
10/29/2026 12:50:54 INFO    export done in 0.1s
```

In [33]:
```python
# The exclusion list: what was kept out, who decided and why (Ignore in the dashboard = source "user")
from src.screen import load_exclusions, EXCLUSIONS
print(EXCLUSIONS)
load_exclusions()
```

```
C:\dev\transcript_intel\data\exclusions.csv
```

Out[33]:

	call_id	title	date	action	source	reason	decided_at
0	01KQ2C53D16CE65DEA612320	Team Celebration - Priya's Birthday	07/02/2026	exclude	user	Ignored in the dashboard (keyword "Celebration")	10/29/2026 19:52 UTC

15. Extending the dashboard

Add a chart. Write a function that takes DataFrames and returns a Plotly figure (pass it through `_style()` for the shared layout), try it in a cell here, then call `st.plotly_chart(fig, width="stretch")` from the right `render_*` function.

Add a tab. Write `render_mytab(ctx)`, add its name to the `st.tabs([...])` list in `main()`, and call it in a new `with tabs[n]:` block. If it needs a new per-period number, add it to `build_context()` so this notebook and the dashboard share it.

Conventions to keep.

- Fixed colours per call type (`CALL_TYPE_COLORS`). Status colours (`GOOD` , `WARNING` , `CRITICAL`) are only for state: risk tiers, SLA breaches, budget.
- Dates as mm/dd/yyyy (`DATE_FMT` , `DATETIME_FMT` , `PLOTLY_DATE`). Tables go through `show_table()` , which also formats date columns and labels internal meetings.
- Plain-language labels: no library or model names in the UI.
- Nothing slow or paid in the app. Anything that needs an LLM or a model belongs in the pipeline, and the app reads its output.

After changing `app.py` : a running `streamlit run` session picks the change up on save (*Rerun* in the top-right corner). The .exe has its own copy of the code and needs a rebuild (`.\build_exe.ps1`).